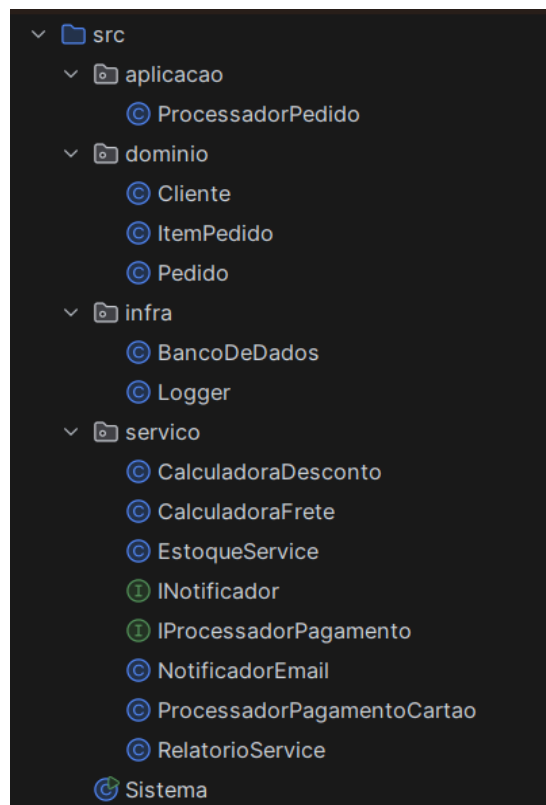


Prática I - Propriedades de Projeto

Pedro Valete, Juliano Tavares, Bernardo Vieira, Gelson Júnior

Resumo da refatoração:

- **Coesão:** Pedido deixou de ser uma “God Class”. Cada responsabilidade (desconto, frete, estoque, pagamento, notificacao, relatorio, banco, log) virou uma classe própria.
- **Acoplamento:** *Pedido* não instancia mais ninguém. *ProcessadorPedido* recebe todos os serviços pelo construtor (injeção de dependência).
- **Ocultamento:** Atributos de *Pedido* viraram private + getters. Listas paralelas (produtos/precos/quantidades) viraram List.
- **Integridade:** *gerarRelatorio()* duplicado foi unificado em *RelatorioService*. *Logger* separado de *BancoDeDados*. Tipo de pagamento deixou de ser string mágica.



1. Coesão

Problema: A classe *Pedido* fazia tudo: gerenciava itens, calculava total, desconto e frete, atualizava estoque, processava pagamento, enviava notificação, gerava relatório e salvava no banco — 8 responsabilidades distintas (God Class).

Solução: Cada responsabilidade foi extraída para uma classe própria:

Antes (tudo em Pedido)	Depois (uma classe por responsabilidade)
<code>aplicarDesconto()</code>	<code>servico.CalculadoraDesconto</code>
<code>calcularFrete()</code>	<code>servico.CalculadoraFrete</code>
<code>atualizarEstoque()</code>	<code>servico.EstoqueService</code>
<code>processarPagamento()</code>	<code>servico.ProcessadorPagamentoCartao</code>
<code>enviarNotificacao()</code>	<code>servico.NotificadorEmail</code>
<code>gerarRelatorio()</code>	<code>servico.RelatorioService</code>
<code>salvarNoBanco()</code>	<code>infra.BancoDeDados + infra.Logger</code>
<code>finalizar()</code>	<code>aplicacao.ProcessadorPedido</code>

Benefício: Cada classe agora tem uma única razão para mudar. Alterar a regra de desconto afeta apenas *CalculadoraDesconto*, sem risco de quebrar pagamento, estoque ou notificação, por exemplo.

2. Acoplamento

Problema: Pedido criava suas dependências diretamente (`new RelatorioService()`) e usava chamadas estáticas (`BancoDeDados.salvarPedido(this)`). Era impossível trocar uma implementação sem alterar Pedido.

Solução: Foi criado um orquestrador (*ProcessadorPedido*) que recebe todos os serviços pelo construtor:

Antes (em Pedido.java):

```
private RelatorioService relatorioService = new RelatorioService();
BancoDeDados.salvarPedido(this);
BancoDeDados.salvarLog("Pedido salvo: " + clienteNome);
processarPagamento("cartao");
```

Depois (em Sistema.java):

```
new ProcessadorPedido(
    new CalculadoraDesconto(),
    new CalculadoraFrete(),
    new EstoqueService(),
    new ProcessadorPagamentoCartao(),
    new NotificadorEmail(),
    new RelatorioService(),
    new BancoDeDados(),
    new Logger()
);
```

Interfaces foram criadas apenas nos pontos com variação real de comportamento (*IProcessadorPagamento* e *INotificador*), onde faz sentido trocar de cartão para boleto ou de email para SMS no futuro.

Benefício: Trocar qualquer serviço passou a ser uma linha em *Sistema.java*, sem tocar em nenhuma outra classe.

3. Ocultamento de Informação

Problema: Todos os 9 atributos de *Pedido* eram public. Qualquer código externo podia alterar o estado interno do pedido sem qualquer validação.

Solução: Todos os atributos viraram private e o acesso de leitura é feito via getters . As 3 listas paralelas frágeis viraram uma única List, onde cada item encapsula nome, preço e quantidade. Os dados do cliente viraram a classe *Cliente*, exigida no construtor de *Pedido*.

Benefício: Estado interno protegido. Impossível desincronizar listas ou criar um pedido sem cliente.

4. Integridade Conceitual

Problemas encontrados:

1. *gerarRelatorio()* presente em *Pedido* e *gerar()* em *RelatorioService* (duas formas, mesma ideia).
2. *BancoDeDados.salvarLog(...)* causa uma inconsistência com o log no lugar não recomendado.
3. *processarPagamento("cartao")* é um problema, pois está totalmente hardcoded.
4. *finalizar()* presente em *Pedido*, inconsistência por estar presente na própria entidade.

Solução:

1. Unificado em *RelatorioService.gerarResumo()* e *RelatorioService.gerarDetalhado()*.
2. Criada a classe própria para log (*Logger.registrar(...)*).
3. Agora com tipo concreto e polimorfismo (*new ProcessadorPagamentoCartao()*).
4. Transferida a responsabilidade para o orquestrador, e não mais na entidade (*finalizar(Pedido)* em *ProcessadorPedido*).

Benefício: O código segue um único padrão. Nomes foram padronizados usando verbos + sufixo (*gerarResumo*, *gerarDetalhado*), sem duplicação, sem código morto. Mais fácil de ler e manter.

